

PromptOps

Phase-Wise Development Blueprint

Agentic DevOps Platform for Project Managers

April 2026 — January 2027 | 9-Month End-to-End Roadmap

Version 1.0 | For Internal Team Use Only

Executive Summary

PromptOps is an Agentic DevOps Utility that allows Project Managers and startup founders to deploy, manage, and heal cloud infrastructure using plain English commands. It replaces the routine work of DevOps Engineers, Cloud Engineers, and Site Reliability Engineers using a Triple-Agent Architecture — the Architect Agent, SRE Agent, and Security Agent — running on AWS, Azure, and GCP.

This document is the single source of truth for the entire engineering team. Every phase is broken into weeks and days, with explicit task lists, assigned owners, AI tools to use, deliverables, and risk mitigations. Nothing in this document is aspirational — every item is actionable and must be completed before the next phase begins.

Product name	PromptOps
Development start	April 7, 2026
Launch target	January 2027
Total duration	9 months (39 weeks)
Primary users	Project Managers, Startup Founders
Cloud targets	AWS (Phase 1–2), Azure + GCP (Phase 3+)
Core model	Human-in-the-loop with AI heavy lifting
Architecture	Triple-Agent: Architect + SRE + Security
Methodology	Agile — 2-week sprints within each phase

How to Read This Document

Each phase section contains: (1) Objectives — what success looks like. (2) Week-by-week task breakdown with day-level detail for critical work. (3) AI Tool Assignments — which specific AI tool handles which task. (4) Deliverables — the exact outputs that mark completion. (5) Risk and corner cases with mitigations. (6) Entry and exit criteria so no phase starts before the previous is complete.

1. System Architecture Overview

Before building anything, every team member must understand how the three agents relate to each other and to the outside world. This is the architectural contract that all code must conform to.

1.1 Triple-Agent Architecture

- **Architect Agent:** Architect Agent
 - Receives structured JSON task lists from the NLP Parser.
 - Generates Terraform or Pulumi Infrastructure-as-Code (IaC).
 - Executes all code through a Shadow Validation sandbox before applying to live cloud.
 - Responsible for: deployments, scaling configs, network topology, IAM policies, cost optimisation.
- **SRE Agent:** SRE Agent
 - Reads Prometheus and CloudWatch metrics every 60 seconds.
 - Uses Causal AI to find root cause, not just symptoms.
 - Generates 3-sentence plain-English RCA for the PM dashboard.
 - Auto-remediates low-risk events (disk full, pod crash, log rotation) without waking anyone.
 - Escalates with full context when human judgment is required.
- **Security Agent:** Security Agent (OPA Guardrail)
 - Non-LLM layer — this is pure deterministic code, not AI.
 - Intercepts every proposed action from the Architect and SRE agents.
 - Checks action against Open Policy Agent (OPA) rules before execution.
 - Blocks and escalates: delete of production-tagged resource, IAM policy with `:*` permissions, port 0.0.0.0/0 open.

1.2 Data Flow (End-to-End)

1. PM types a plain-English command into the PromptOps UI.
2. NLP Parser (Claude Sonnet 4 + LangGraph) converts it to structured JSON intent.
3. JSON routed to the relevant agent (Architect for infra, SRE for monitoring).
4. Agent generates a proposed action (IaC code, remediation script, scaling event).
5. Security Agent (OPA) intercepts and checks the proposed action against policy rules.
6. If blocked: PM receives an approval card with full context. No action taken until PM approves.
7. If approved: Shadow Validation runs the action in a sandbox environment first.
8. If sandbox passes: Action applied to live cloud with atomic rollback armed.
9. If sandbox fails: Agent revises and retries. PM notified if 3 retries fail.
10. Result displayed on PM Dashboard with plain-English summary.
11. All actions logged immutably to audit trail (S3 + DynamoDB).

1.3 Non-Negotiable Engineering Constraints

- Every destructive action (DELETE, TERMINATE, DETACH) requires explicit PM approval with resource name typed back.
- No action is ever applied to prod without passing the shadow validation sandbox first.

- Terraform state files are version-snapshotted before every apply. Corruption triggers auto-restore, never re-apply.
- Auto-remediation circuit breaker: if same service is restarted more than 3 times in 10 minutes, all automation halts.
- All external text (logs, ticket descriptions) is sanitised before being fed to any LLM — prompt injection defence.
- Ambiguity scoring: if LLM confidence is below 85%, action is blocked and PM is asked to clarify.
- Every action carries an immutable timestamp, PM approval record, and agent decision log.

Phase 1: Foundation & NLP Intent Engine

April 7 — June 27, 2026 | 12 Weeks | Sprint 1–6

Phase 1 Objective

Build the brain of PromptOps — the Natural Language Parser (NLP) that converts a PM's plain-English command into a structured, machine-executable JSON task list without hallucinations. This is the foundation that all three agents depend on. Nothing in Phase 2 can start until Phase 1 is complete and passing accuracy tests.

Phase 1 Entry Criteria

- AWS sandbox account provisioned (free tier is acceptable at this stage).
- Development environment set up: Node.js 22, Python 3.11, Docker, Terraform CLI, Pulumi CLI.
- GitHub repo created with branch protection on main. No direct pushes to main.
- Linear project created: "PromptOps Launch" with Phase 1 cycle active.
- All team members have Claude Sonnet 4 API access via Anthropic Platform.

Phase 1 Week-by-Week Plan

Week 1–2 (Apr 7–18): User Intent Research & Command Library

Goal: Understand the vocabulary of Project Managers. Before writing a single line of parser code, we must collect real PM language from real sources.

Task	Owner	AI Tool	Duration
Scrape and collect 200+ real PM requests from public Jira/GitHub issues and PM forums	Research Lead	Claude Sonnet 4 (batch analysis)	3 days
Classify all collected requests into 8 intent categories: deploy, scale, rollback, monitor, audit, cost, security, diagnose	NLP Engineer	Claude Sonnet 4	2 days
Build Command Library JSON schema with fields: intent_type, target_service, target_env, parameters, confidence_floor	Backend Engineer	Cursor Agent Mode	2 days
Write 50 "golden test" PM commands that the NLP must parse correctly — these become the regression suite	QA Engineer	Claude Sonnet 4	3 days
Set up LangGraph agent framework: nodes, edges, state management, retry logic	Backend Engineer	LangGraph + Cursor	2 days

Week 3–4 (Apr 21–May 2): NLP Parser v1 — Intent-to-JSON

Goal: Build the first version of the parser that converts PM text to structured JSON.

Task	Owner	AI Tool	Duration
Write the system prompt for Claude Sonnet 4 that enforces strict JSON output with no prose	NLP Engineer	Claude Sonnet 4 API	2 days
Build ambiguity detection: if confidence < 85%, return a clarification card instead of guessing	NLP Engineer	Claude Sonnet 4	2 days
Build the clarification card UI component: "I found 3 services matching X. Which one?"	Frontend Engineer	Cursor Agent Mode	2 days
Implement input sanitisation layer: strip prompt injection patterns before LLM call	Security Engineer	Custom regex + OPA rules	2 days
Run all 50 golden tests. Target: >90% accuracy. Document failures.	QA Engineer	Claude Sonnet 4 (eval)	3 days

Week 5–6 (May 5–16): Task Decomposition Engine

Goal: For each parsed intent, decompose it into 5–15 DevOps sub-tasks that can be executed by the Architect Agent.

Task	Owner	AI Tool	Duration
Design the decomposition prompt: one PM command → ordered list of sub-tasks with dependencies	NLP Engineer	Claude Sonnet 4 API	3 days
Build the sub-task JSON schema: task_id, action_type, resource, parameters, depends_on, rollback_action	Backend Engineer	Cursor Agent Mode	2 days
Implement dependency graph resolver: tasks execute in correct order, parallelise where safe	Backend Engineer	LangGraph	3 days
Test decomposition for 10 complex commands: "Deploy 3-tier app with RDS and auto-scaling on AWS"	QA Engineer	Claude Sonnet 4	2 days
Build the task preview UI: PM sees a step-by-step list of what will happen before approving	Frontend Engineer	Cursor Agent Mode	2 days

Week 7–8 (May 19–May 30): Context & Memory Layer

Goal: PromptOps must remember the state of the customer's infrastructure between conversations so it doesn't ask the same questions twice.

Task	Owner	AI Tool	Duration
Design the Infrastructure Context Store: a JSON snapshot of all known resources, tags, and relationships	Backend Engineer	Claude Sonnet 4 + DynamoDB	3 days
Build the context injection pipeline: auto-attach relevant context to every LLM call	Backend Engineer	LangGraph	2 days
Implement context refresh: re-scan AWS account every 15 minutes to detect manual changes (drift detection)	Backend Engineer	Terraform + AWS SDK	3 days
Build the drift detection alert: "Someone changed X manually. Codify or revert?"	Frontend Engineer	Cursor Agent Mode	2 days
Test: PM says "Scale the API" — system should know which API without asking (from context)	QA Engineer	Claude Sonnet 4	2 days

Week 9–10 (Jun 2–13): PM Dashboard Shell

Goal: Build the front-end interface that PMs will use. This is the only surface the PM ever touches.

Task	Owner	AI Tool	Duration
Design the dashboard layout: command input, task preview panel, incident cards, audit trail	Frontend Engineer	Cursor Agent Mode + v0.dev	3 days
Build the command input component with real-time intent preview (shows parsed intent as PM types)	Frontend Engineer	Cursor Agent Mode	3 days
Build the approval flow UI: shows exactly what will happen, requires PM confirmation for risky actions	Frontend Engineer	Cursor Agent Mode	2 days
Build the audit trail view: every action, timestamp, PM approval, agent decision visible	Frontend Engineer	Cursor Agent Mode	2 days
Mobile responsiveness: dashboard must work on phone for 3 AM on-call approvals	Frontend Engineer	Cursor Agent Mode	2 days

Week 11–12 (Jun 16–27): Phase 1 Integration & Testing

Goal: Wire all Phase 1 components together and prove the end-to-end NLP pipeline works reliably.

Task	Owner	AI Tool	Duration
Integration test: PM command → NLP Parser → JSON → Task Preview on dashboard	QA Engineer	Claude Sonnet 4 (eval)	3 days

Task	Owner	AI Tool	Duration
Run full regression suite: all 50 golden test commands must pass at >92% accuracy	QA Engineer	Claude Sonnet 4 (eval)	2 days
Adversarial testing: try to confuse the parser with ambiguous, contradictory, and malicious inputs	Security Engineer	Claude Sonnet 4	2 days
Performance test: NLP Parser must return JSON within 3 seconds for 95% of requests	QA Engineer	k6 load testing	2 days
Phase 1 demo to stakeholders: live walkthrough of 5 different PM commands end-to-end	Lead Engineer	—	1 day

Phase 1 AI Tool Summary

Claude Sonnet 4	Primary LLM for all NLP parsing, decomposition, and evaluation tasks. Use claude-sonnet-4-20250514 via Anthropic API.
LangGraph	Agent orchestration framework. Manages multi-step flows, state, retries, and parallel task execution.
Cursor Agent Mode	AI-powered IDE for building backend modules and frontend components autonomously. Engineers review, not write.
v0.dev	Rapid UI component generation for the PM dashboard. Feed it design requirements, get React components.
OPA (Open Policy Agent)	Non-LLM policy engine. All security guardrail rules written in Rego. No AI involved in safety decisions.

Phase 1 Deliverables (Exit Criteria)

- NLP Parser achieving >92% accuracy on 50 golden test commands.
- Command Library JSON schema with 8 intent categories and 50+ examples.
- Task decomposition working for 10 complex multi-step PM commands.
- Infrastructure Context Store connected to AWS account with 15-minute drift detection.
- PM Dashboard shell: command input, task preview, approval flow, audit trail.
- Performance: NLP returns JSON in under 3 seconds at 100 concurrent requests.
- Security: all 10 prompt injection test vectors blocked by input sanitisation.

Phase 1 Risks & Mitigations

Risk / Corner Case	Severity	Mitigation Strategy
LLM returns malformed JSON breaking the downstream pipeline	High	Enforce JSON Schema validation on every LLM response. Retry up to 3 times with error message in context. Fallback to asking PM to rephrase.
Ambiguity scoring misses a dangerous misinterpretation	Extreme	All actions that touch "prod", "production", or "live" tagged resources require explicit PM confirmation regardless of confidence score.

Risk / Corner Case	Severity	Mitigation Strategy
Context store grows too large for LLM context window	Medium	Context is chunked. Only resources relevant to the current command are injected. Full context available on explicit request only.
Prompt injection via PM input	Extreme	Input sanitisation layer strips all instruction-pattern text before LLM call. PM input is wrapped in delimiters that signal "this is user data, not instructions."

Phase 2: Architect Agent — Infrastructure-as-Code Engine

June 30 — September 26, 2026 | 13 Weeks | Sprint 7–13

Phase 2 Objective

Build the Architect Agent that converts the structured JSON from the NLP Parser into real, working cloud infrastructure on AWS. Every line of infrastructure code is generated by AI, validated in a sandbox before touching live systems, and applied atomically so any failure triggers a complete rollback. By the end of Phase 2, a PM must be able to type "Deploy a 3-tier web app with auto-scaling and RDS" and have it running on AWS within 10 minutes.

Phase 2 Week-by-Week Plan

Week 13–14 (Jun 30–Jul 11): IaC Code Generator v1

Task	Owner	AI Tool	Duration
Build the Terraform code generation prompt: takes sub-task JSON, outputs valid HCL Terraform	Backend Engineer	Pulumi AI + Claude Sonnet 4	4 days
Implement code generation for 5 core AWS resource types: EC2, VPC, RDS, S3, IAM roles	Backend Engineer	Pulumi AI	4 days
Build the Pulumi AI integration as secondary generator for complex multi-resource stacks	Backend Engineer	Pulumi AI SDK	3 days
Write the IaC validation schema: generated code must pass terraform validate before proceeding	DevOps Engineer	Terraform CLI	2 days
Unit test: generate Terraform for each of the 5 resource types, validate, check for common mistakes	QA Engineer	Claude Sonnet 4 (eval)	3 days

Week 15–16 (Jul 14–25): Shadow Validation Sandbox

The shadow validation sandbox is the most critical safety layer in Phase 2. No generated code ever touches the live cloud without passing here first.

Task	Owner	AI Tool	Duration
Provision the shadow sandbox: isolated AWS account with no production resources, full VPC	DevOps Engineer	Terraform + AWS CLI	2 days

Task	Owner	AI Tool	Duration
Build the dry-run pipeline: apply generated Terraform to sandbox, run health checks, capture full output log	Backend Engineer	Terraform + AWS SDK	4 days
Implement health check suite: resource created, ports reachable, IAM policy valid, no unexpected costs	Backend Engineer	Python + AWS SDK	3 days
Build the revision loop: if sandbox fails, feed error back to Claude Sonnet 4 for code correction, retry up to 3 times	Backend Engineer	Claude Sonnet 4 API	3 days
Test: deliberately introduce 10 types of Terraform errors and verify the revision loop corrects all of them	QA Engineer	Claude Sonnet 4	3 days

Week 17–18 (Jul 28–Aug 8): Atomic Deployment & Rollback

Every deployment is a transaction. Either all resources are created successfully, or none are. There is no partial state.

Task	Owner	AI Tool	Duration
Implement Terraform state locking via S3 + DynamoDB: only one deployment can run at a time	Backend Engineer	Terraform + AWS SDK	2 days
Build state file versioning: snapshot the Terraform state to S3 before every apply	Backend Engineer	AWS SDK	2 days
Implement the atomic rollback trigger: if any health check fails post-apply, auto-restore previous state snapshot	Backend Engineer	Terraform + Python	3 days
Build the half-deployed state detector: if apply is interrupted, flag the environment as "broken" and halt all further automation	Backend Engineer	Python + AWS SDK	3 days
Test: simulate 5 deployment interruptions (network timeout, IAM error, resource limit) and verify clean rollback every time	QA Engineer	AWS Fault Injection	3 days

Week 19–20 (Aug 11–22): IAM, Networking & Security Configs

Task	Owner	AI Tool	Duration
Build least-privilege IAM policy generator: generates minimal permissions per service, never uses *.* wildcard	Security Engineer	Claude Sonnet 4 + OPA	4 days
Build VPC and security group generator: creates correct subnet topology, NACL rules, security group chains	Backend Engineer	Pulumi AI	4 days

Task	Owner	AI Tool	Duration
Implement the OPA guardrail rules for networking: block port 22 open to 0.0.0.0/0, block 0.0.0.0/0 on all ports	Security Engineer	OPA Rego	2 days
Implement the OPA guardrail rules for IAM: block AdministratorAccess, block *, flag any cross-account trust	Security Engineer	OPA Rego	2 days
Test: attempt to generate 15 known-bad IAM and network configs and verify OPA blocks all of them	QA Engineer	OPA + custom test suite	3 days

Week 21–22 (Aug 25–Sep 5): Cost Optimisation Engine

Task	Owner	AI Tool	Duration
Build the resource tagging enforcer: every created resource must have env, team, cost-centre tags or deploy is blocked	Backend Engineer	OPA + Terraform	2 days
Build the cost estimation preview: before applying, show PM estimated monthly cost for the proposed infrastructure	Backend Engineer	AWS Pricing API + Claude Sonnet 4	4 days
Implement unused resource scanner: weekly scan for idle EC2, unattached EBS, empty S3 buckets older than 30 days	Backend Engineer	AWS SDK + Python	3 days
Build cost saving card: "Delete this idle DB to save ₹50,000/month — Approve?" shown on PM dashboard	Frontend Engineer	Cursor Agent Mode	2 days
Implement cost ceiling guardrail: any single action that would increase monthly bill by >20% requires PM approval	Security Engineer	OPA + AWS Pricing API	2 days

Week 23–25 (Sep 8–26): Phase 2 Integration & End-to-End Testing

Task	Owner	AI Tool	Duration
Integration test: full flow from PM command → NLP → Architect Agent → Shadow Validation → Live AWS deploy	QA Engineer	Claude Sonnet 4 + k6	4 days
Test the 5 canonical PM commands: deploy 3-tier app, scale service, rollback deployment, add RDS, configure VPC	QA Engineer	Claude Sonnet 4	3 days
Performance test: complete deployment of a 5-resource stack must finish in under 10 minutes end-to-end	QA Engineer	k6	2 days
Security penetration test: attempt to bypass OPA guardrails via crafted PM inputs, malformed JSON, API direct calls	Security Engineer	OWASP ZAP + manual	3 days

Task	Owner	AI Tool	Duration
Phase 2 demo: live deploy of a 3-tier web app from a single PM command in front of stakeholders	Lead Engineer	—	1 day

Phase 2 AI Tool Summary

Pulumi AI	Primary IaC generator. Takes intent JSON, outputs multi-cloud resource definitions. Best for complex stacks.
Claude Sonnet 4	Secondary IaC generator for Terraform HCL. Also used for revision loop when sandbox validation fails.
Cursor Agent Mode	Builds backend modules (rollback engine, cost scanner, drift detector) autonomously. Engineers review output.
OPA (Rego)	All security guardrail rules. Non-LLM. Rules written by Security Engineer, not generated by AI.
AWS Pricing API	Cost estimation before deploy. Claude Sonnet 4 formats the cost breakdown into PM-readable English.
AWS Fault Injection Simulator	Simulates infrastructure failures for rollback testing. Validates atomic rollback under real failure conditions.

Phase 2 Deliverables (Exit Criteria)

- Architect Agent generates valid Terraform for 10+ AWS resource types without errors.
- Shadow Validation passes before every live deployment — zero exceptions.
- Atomic rollback tested successfully for 5 different failure types.
- OPA guardrails block 100% of tested bad IAM and networking configurations.
- Cost estimation shown to PM before every deployment with >90% accuracy vs actual bill.
- 5 canonical PM commands deploy successfully end-to-end in under 10 minutes each.

Phase 2 Risks & Mitigations

Risk / Corner Case	Severity	Mitigation Strategy
Terraform state file corruption mid-apply	Extreme	State versioned to S3 before every apply. Corruption triggers auto-restore from last clean snapshot. Never re-apply without human review.
Shadow sandbox costs accumulate	Medium	Sandbox uses spot instances, auto-terminates all resources after each test within 30 minutes. Cost ceiling alarm at \$50/month on the sandbox account.
IaC drift: manual changes bypass PromptOps	High	15-minute drift detection scan. Drift surfaces as a PM card. Manual changes are never silently overwritten — PM must choose: codify or revert.
OPA rule conflicts with legitimate use case	Medium	OPA rules have an override mechanism: PM can request an exception which creates an audit-logged policy exception. No permanent rule removal.

Phase 3: SRE Agent — Predictive Monitoring & Self-Healing

September 29 — November 21, 2026 | 8 Weeks | Sprint 14–17

Phase 3 Objective

Build the SRE Agent that eliminates on-call burnout. It monitors infrastructure 24/7 without fatigue, predicts failures 24 hours before they happen, auto-resolves 80% of incidents without waking anyone, and delivers plain-English Root Cause Analysis with a one-tap fix for anything it escalates. By the end of Phase 3, the PM should be able to sleep through the night.

Phase 3 Week-by-Week Plan

Week 26–27 (Sep 29–Oct 10): Monitoring Integration & Baseline Learning

Task	Owner	AI Tool	Duration
Connect SRE Agent to Prometheus: pull metrics for CPU, memory, disk, network, latency, error rate every 60 seconds	Backend Engineer	Prometheus SDK + Python	3 days
Connect SRE Agent to AWS CloudWatch: pull EC2, RDS, Lambda, ELB metrics in real time	Backend Engineer	AWS SDK + Python	2 days
Implement 14-day baseline learning period: SRE Agent observes and records normal traffic patterns before enabling auto-remediation	Backend Engineer	Python + DynamoDB	3 days
Build dynamic threshold engine: thresholds are set from baseline data, not manual config. Auto-adjusts as patterns change.	Backend Engineer	Python + Claude Sonnet 4	3 days
Build alarm fatigue detector: if >3 false positives per day, auto-increase threshold sensitivity and alert team	Backend Engineer	Python	2 days

Week 28–29 (Oct 13–24): Causal AI Root Cause Analysis

This is the core of the SRE Agent. Standard monitoring shows THAT something is wrong. Causal AI shows WHY it started. These are fundamentally different.

Task	Owner	AI Tool	Duration
Integrate Dynatrace Davis AI API: connect to deployment event stream, log stream, and metric stream simultaneously	Backend Engineer	Dynatrace API + Python	4 days

Task	Owner	AI Tool	Duration
Build the RCA synthesis prompt: takes Davis AI output + raw metrics, generates 3-sentence PM summary	NLP Engineer	Claude Sonnet 4 API	3 days
Build the timeline correlation engine: maps deployment events to metric changes to pinpoint the cause commit	Backend Engineer	Python + Dynatrace API	3 days
Build the RCA card UI component: What happened, Why it happened, Proposed fix, Confidence score, Approve button	Frontend Engineer	Cursor Agent Mode	3 days
Test RCA accuracy: introduce 10 known failure scenarios and verify the agent correctly identifies root cause in each	QA Engineer	AWS Fault Injection	3 days

Week 30–31 (Oct 27–Nov 7): Auto-Remediation Engine

Task	Owner	AI Tool	Duration
Build auto-remediation playbook library: 20 common incidents mapped to specific automated fix scripts	Backend Engineer	Python + AWS SDK	4 days
Implement circuit breaker: if same service restarted >3 times in 10 min, halt all automation and escalate immediately	Backend Engineer	Python	2 days
Build the cascading failure detector: if fix of A causes failure of B, halt and escalate — never enter a remediation loop	Backend Engineer	Python + LangGraph	3 days
Build the "Saved You" counter on PM dashboard: shows total incidents auto-resolved, cost saved, downtime prevented	Frontend Engineer	Cursor Agent Mode	2 days
Test the circuit breaker: simulate a cascading restart loop and verify automation halts at the 3rd restart	QA Engineer	AWS Fault Injection	3 days

Week 32–33 (Nov 10–21): SLO Tracking, Predictive Alerts & Post-Mortems

Task	Owner	AI Tool	Duration
Build SLO tracking dashboard: shows current uptime, error budget, burn rate for each service	Frontend Engineer	Cursor Agent Mode	3 days
Implement error budget burn rate predictor: "At current rate, you breach 99.9% SLO in 18 hours. Here is the fix."	Backend Engineer	Claude Sonnet 4 + Python	3 days

Task	Owner	AI Tool	Duration
Build predictive alert system: uses 14-day baseline to forecast traffic spikes 24 hours in advance and pre-scale	Backend Engineer	Claude Sonnet 4 + Python	3 days
Build auto-post-mortem generator: on incident close, generate full timeline, root cause, blast radius, and action items	Backend Engineer	Claude Sonnet 4 API	2 days
Test predictive alerts: replay historical traffic data and verify the system would have pre-scaled before each spike	QA Engineer	Python + historical data	3 days

Phase 3 AI Tool Summary

Dynatrace Davis AI	Causal AI engine for root cause analysis. Identifies WHY a failure started, not just that it exists.
Claude Sonnet 4	Translates Davis AI technical output into 3-sentence PM-readable RCA. Also powers predictive forecasting.
LangGraph	Orchestrates the multi-step remediation flows with state tracking and circuit breaker integration.
Sherlocks.ai (optional)	Deep log analysis for complex incidents where Dynatrace data is insufficient.
AWS Fault Injection	Simulates real infrastructure failures to test auto-remediation playbooks before going live.
Cursor Agent Mode	Builds all monitoring dashboard UI components and the auto-post-mortem interface.

Phase 3 Deliverables (Exit Criteria)

- SRE Agent monitors Prometheus and CloudWatch with 60-second polling — no gaps.
- Causal RCA correctly identifies root cause for 9 out of 10 simulated failure scenarios.
- Auto-remediation resolves at least 8 out of 10 common incident types without human intervention.
- Circuit breaker halts automation correctly in cascading failure simulation — 100% of tests.
- Predictive alerts fire at least 20 hours before 80% of simulated traffic spikes in replay testing.
- Auto-post-mortem generated within 60 seconds of incident closure for every tested scenario.
- PM sleeps through 80% of simulated on-call events in a 1-week load test scenario.

Phase 3 Risks & Mitigations

Risk / Corner Case	Severity	Mitigation Strategy
Auto-remediation creates a cascading restart loop	Extreme	Circuit breaker halts at 3rd restart in 10 minutes. No exceptions. Human must manually release the circuit breaker from the dashboard.
Prompt injection via logs feeds malicious instruction to SRE Agent	Extreme	All log text is sanitised and wrapped in delimiters before LLM processing. Log content can never

Risk / Corner Case	Severity	Mitigation Strategy
		trigger an action directly — only structured metric data can.
False positive predictive alert causes unnecessary pre-scaling cost	High	Cost ceiling on any auto-scaling action. If action would cost more than 15% above baseline, PM approval required before execution.
Dynatrace API rate limits during high-incident period	Medium	Local metric cache stores last 60 minutes of data. If API is unavailable, SRE Agent continues with cached data and flags the gap.

Phase 4: Chaos Engineering & Scale Testing

November 24 — December 26, 2026 | 5 Weeks | Sprint 18–19

Phase 4 Objective

Prove that PromptOps can handle Amazon/Flipkart-sale level traffic — 10x normal load — and self-heal when components fail. Every feature must pass the chaos test suite before it is considered production-ready. The deliverable from Phase 4 is a certified "1 Million User" load test report and a Chaos Engineering scorecard, both of which become core sales assets.

Phase 4 Week-by-Week Plan

Week 34–35 (Nov 24–Dec 5): Load & Stress Testing

Task	Owner	AI Tool	Duration
Write k6 load test scripts for 5 core user journeys: deploy, scale, rollback, monitor, approve RCA fix	QA Engineer	k6 + Grafana Cloud	4 days
Run graduated load tests: 100, 1,000, 10,000, 100,000, 1,000,000 virtual users. Document breaking points.	QA Engineer	k6 + BlinqIO	4 days
Stress test the NLP Parser: 1,000 concurrent PM commands. Must return JSON in under 3 seconds for 95%.	QA Engineer	k6	2 days
Stress test the Architect Agent: 100 concurrent IaC generation requests. No state corruption or race conditions.	QA Engineer	k6 + AWS	3 days
Fix all performance bottlenecks discovered. Re-run until all benchmarks pass.	Backend Engineers	Cursor Agent Mode	variable

Week 36–37 (Dec 8–19): Chaos Engineering

Task	Owner	AI Tool	Duration
Set up Chaos Mesh on the Kubernetes cluster: randomised pod killing, network partition, CPU throttle	DevOps Engineer	Chaos Mesh	3 days
Test: kill any random pod — SRE Agent must detect and restart within 60 seconds, 100% of trials	QA Engineer	Chaos Mesh + LitmusChaos	3 days
Test: simulate network partition between services — system must degrade gracefully, not corrupt data	QA Engineer	Chaos Mesh	2 days

Task	Owner	AI Tool	Duration
Test: simulate disk full on prod server — SRE Agent must clear space and alert PM before service degrades	QA Engineer	LitmusChaos	2 days
Test: inject bad Terraform code — sandbox must catch it, rollback must trigger, PM must be notified within 2 minutes	QA Engineer	Custom test suite	2 days
Regression gate implementation: no feature ships to prod unless it passes the full chaos suite automatically via CI/CD	DevOps Engineer	GitHub Actions + Chaos Mesh	3 days

Week 38–39 (Dec 22–26): Final Hardening & Launch Preparation

Task	Owner	AI Tool	Duration
Security penetration test: full external pentest of the PromptOps API and PM dashboard	Security Engineer	OWASP ZAP + manual	3 days
Cloud Readiness Audit feature: scans customer AWS account and produces a health score and readiness report	Backend Engineer	Claude Sonnet 4 + AWS SDK	4 days
Shadow Mode implementation: read-only trial mode where PromptOps suggests actions but never executes them	Backend Engineer	Cursor Agent Mode	2 days
Legal review: Terms of Service for AI liability, professional liability insurance, data processing agreement	Legal / Founder	External legal counsel	5 days
Launch readiness checklist: all Phase 1–4 exit criteria verified. First customer demo environment prepared.	All team leads	—	2 days

Phase 4 AI Tool Summary

k6 (Grafana Cloud)	Industry-standard load testing. Scales from 10 to 1,000,000 virtual users. Native Grafana dashboards.
BlinqIO	AI virtual testers that discover bugs 24/7 without manual test scripts. Runs after every deployment.
Chaos Mesh	Kubernetes-native chaos engineering. Pod kill, network partition, CPU throttle injection.
LitmusChaos	Chaos engineering for infrastructure-level faults: disk full, node failure, cloud region outage simulation.
Claude Sonnet 4	Powers the Cloud Readiness Audit: analyses customer AWS account and writes the health report.
OWASP ZAP	Open-source security scanner for API and web dashboard penetration testing.

Phase 4 Deliverables (Exit Criteria)

- "1 Million User" certified load test report with p95 latency and error rate documented.
- Chaos Engineering scorecard: 99.9% uptime maintained across all chaos scenarios.
- Pod restart time under 60 seconds verified across 50 trials — 100% success rate.
- Regression gate active in CI/CD: no PR merges without passing chaos suite.
- Cloud Readiness Audit feature producing accurate health scores for 5 test customer environments.
- Shadow Mode working: read-only trial available for first customer demos.
- Legal: Terms of Service reviewed and signed off. Professional AI liability insurance confirmed.
- Security: external pentest complete with all critical and high findings resolved.

Master AI Tool Index

This is the definitive list of every AI tool used in PromptOps development, what it does, and which phase it is needed in. Every team member should be set up with access to the tools for their phase before that phase begins.

AI Tool	Phase	What it does in PromptOps
Claude Sonnet 4 (claude-sonnet-4-2025 0514)	1–4	NLP parsing, task decomposition, IaC code revision, RCA synthesis, predictive alerts, post-mortem generation, Cloud Readiness Audit. Primary intelligence layer.
LangGraph	1–3	Multi-step agent orchestration. State management, retry logic, parallel execution, circuit breaker integration.
Pulumi AI	2	Intent-to-Infrastructure code generation. Best for complex multi-resource AWS stacks.
Cursor (Agent Mode)	1–4	AI-powered IDE. Used by all engineers to build backend modules and frontend components autonomously.
v0.dev	1	Rapid PM dashboard UI component generation. Feed design requirements, get production-ready React.
Dynatrace Davis AI	3	Causal AI for root cause analysis. Identifies WHY an incident started, not just symptoms.
Sherlocks.ai	3	Deep log analysis for complex incidents. Supplements Dynatrace for log-heavy RCA scenarios.
k6 (Grafana Cloud)	4	Load and stress testing. Scales to 1,000,000 virtual users with native Grafana dashboards.
BlinqIO	4	AI virtual testers. Discovers bugs 24/7 without manual test scripts. Runs on every deployment.
Chaos Mesh	4	Kubernetes chaos engineering. Pod kill, network partition, CPU throttle.
LitmusChaos	4	Infrastructure chaos. Disk full, node failure, cloud region outage simulation.
OPA (Open Policy Agent)	2–4	Non-LLM security guardrail layer. All safety rules written in Rego. Intercepts every agent action.
AWS Fault Injection Simulator	2–3	Simulates real infrastructure failures for rollback and auto-remediation testing.
OWASP ZAP	4	API and web security scanner for final penetration testing before launch.

Complete Risk & Corner Case Register

Every risk identified across all phases with its severity and mitigation strategy. This register must be reviewed at the start of every sprint.

Technical & Engineering Risks

Risk / Corner Case	Severity	Mitigation Strategy
AI hallucinates and deletes the wrong production resource	Extreme	OPA intercepts every DELETE. Production-tagged resources require PM to type the exact resource name back. No exceptions.
Half-deployed state leaves infra broken and un-rollbackable	Extreme	Atomic deployment: all-or-nothing. Any failure triggers full rollback to last known good state snapshot.
Terraform state file corrupted mid-apply	Extreme	State versioned to S3 before every apply. Corruption triggers restore, never re-apply without human.
Prompt injection via PM input or log content	Extreme	Input sanitisation strips instruction patterns. External text wrapped in delimiters, never interpreted as instructions.
AI confidence too high on ambiguous PM command	High	If confidence < 85%, PM is shown a clarification card. Action is blocked until PM selects the correct resource.
Multi-cloud config contradiction creates security hole	High	AWS only in Phase 1–2. Conflict Resolver layer added before multi-cloud in Phase 3. Manual PM resolution required on all conflicts.
Cascading auto-remediation restart loop	Extreme	Circuit breaker: >3 restarts of same service in 10 minutes halts all automation. Human must manually release.
Legacy spaghetti infra confuses the AI	Medium	Cloud Readiness Audit assigns confidence score per resource. Below 70% = quarantined. AI will not touch it.
SRE Agent false positive causes alarm fatigue	Medium	Alarm fatigue score tracked. >3 false positives per day triggers auto-threshold adjustment and team alert.
Clock drift causes predictive alerts to fire at wrong time	High	All scheduling in UTC internally. NTP sync monitored. Timezone is required field at onboarding — never inferred.
IaC drift from manual console changes	High	15-min drift scan. Drift surfaced as PM card. Manual changes never silently overwritten.

Sales & Business Risks

Risk / Corner Case	Severity	Mitigation Strategy
CTO refuses write access at final sign-off	Extreme	Shadow Mode as first product: 30 days read-only. AI suggests, never executes. Trust earned through 100 correct suggestions, not a pitch.
AWS Copilot or Datadog AI launches competing feature	High	PM-obsession moat. Counter-demo shows non-technical PM using PromptOps in 60 seconds vs AWS needing IAM setup. Own the PM persona.
Customer churns after one visible AI mistake	High	Cost ceiling guardrail on all auto-actions. "Saved You" counter makes 200 silent successes visible. One mistake is never the whole story.
Pricing rejected vs existing tooling spend	Medium	Stack consolidation pitch. PromptOps replaces PagerDuty + parts of Datadog + parts of Terraform Cloud + 30% engineer time. ROI calculator in demo.

Legal & Operational Risks

Risk / Corner Case	Severity	Mitigation Strategy
Data breach caused by PromptOps action — liability unclear	Extreme	Immutable audit log with PM approval timestamps. ToS limits liability to monthly subscription. Professional AI Liability Insurance required before first enterprise sale.
Customer's legacy infra is too messy for safe AI management	High	Cloud Readiness Audit runs first. Produces a health score and a "Fix these 5 things before we take over" report. Sets expectations before contract.
AI makes a high-cost mistake — customer sues	High	Cost ceiling guardrail on every automated action. Any action increasing monthly bill by >20% requires PM approval. ToS caps consequential damages.

Master Timeline at a Glance

Phase	Period	Duration	Primary Deliverable
Phase 1	Apr 7 – Jun 27	12 weeks	NLP Parser + Command Library + PM Dashboard shell. >92% accuracy on 50 golden tests.
Phase 2	Jun 30 – Sep 26	13 weeks	Architect Agent: full IaC generation + shadow validation + atomic rollback + OPA guardrails.
Phase 3	Sep 29 – Nov 21	8 weeks	SRE Agent: causal RCA + auto-remediation + predictive alerts + auto-post-mortems.
Phase 4	Nov 24 – Dec 26	5 weeks	"1M User" load test + Chaos scorecard + Shadow Mode + Cloud Readiness Audit + Legal sign-off.
Launch	January 2027	—	First paying customer onboarded. Product is ready-to-sell.

Week 1 Immediate Action Items

The following must be completed in the first 5 working days before any development starts:

- 12. Provision AWS sandbox account (free tier). Share credentials with all engineers via 1Password or AWS IAM Identity Center.
- 13. Set up GitHub organisation: create "promptops" repo, enable branch protection on main, set up PR review requirements (minimum 1 reviewer).
- 14. Create Linear project "PromptOps Launch" with 4 cycles (one per phase). Input Phase 1 Week 1 tasks and assign owners today.
- 15. All engineers obtain Anthropic API keys for Claude Sonnet 4. Set up rate limit monitoring from day one.
- 16. Install development tools on all machines: Node.js 22, Python 3.11, Docker Desktop, Terraform CLI 1.9, Pulumi CLI 3.x, k6.
- 17. Conduct a 2-hour team kickoff: walk through this document section by section. Every engineer must understand all four phases, not just their own.
- 18. Set up Slack channels: #promptops-build (daily updates), #promptops-incidents (any prod issue), #promptops-decisions (architectural choices).